

Билет 12. Механизм `io_uring`.

Многопоточный сервер через `io_uring`.

Преимущества и недостатки по сравнению с `epoll`

0. Формулировка билета

Механизм `io_uring`. Реализация многопоточного сервера через `io_uring`. В чём преимущества и недостатки по сравнению с `epoll`?

Главная идея:

`epoll` сообщает о готовности fd: “можно читать/писать”. После этого пользовательский код сам вызывает `read/write`.

`io_uring` позволяет отправлять в ядро конкретные операции: “прочитай”, “запиши”, “прими соединение”. Потом ядро возвращает результат завершённой операции.

Ключевая разница:

```
epoll = readiness-based I/O
      fd готов → пользователь сам вызывает read/write

io_uring = completion-based I/O
      пользователь отправил операцию → ядро позже вернуло результат
```

Или коротко:

```
epoll    → reactor
io_uring → proactor
```

1. Зачем появился `io_uring`

В `epoll`-сервере мы делали примерно так:

```
epoll_wait
получили fd, готовый к чтению
read(fd)
```

```
получили fd, готовый к записи  
write(fd)
```

Это уже намного лучше busy loop polling, потому что мы не обходим все fd вручную.

Но всё ещё есть проблемы:

```
epoll сообщает только готовность, а не завершение операции  
после события всё равно нужен read/write syscall  
для многих операций получается много переходов user ↔ kernel  
сложно унифицировать сеть, файлы, таймеры, accept, send, recv  
нужно вручную управлять состоянием операций
```

`io_uring` пытается уменьшить число системных вызовов и сделать интерфейс более асинхронным.

Идея:

```
пользователь кладёт операции в очередь заявок  
ядро выполняет их  
ядро кладёт результаты в очередь завершений
```

2. Главные структуры `io_uring`

У `io_uring` есть две основные кольцевые очереди:

```
SQ – Submission Queue  
CQ – Completion Queue
```

Submission Queue

Очередь заявок.

Пользователь кладёт туда операции:

```
accept  
read  
write  
recv  
send  
timeout  
close  
connect
```

Каждая заявка описывается структурой:

```
SQE — Submission Queue Entry
```

То есть SQE — это “что я хочу, чтобы ядро сделало”.

Completion Queue

Очередь завершений.

Ядро кладёт туда результаты выполненных операций.

Каждое завершение описывается структурой:

```
CQE — Completion Queue Entry
```

То есть CQE — это “операция завершилась, вот её результат”.

Схема:

```
user space:
  SQE: "read fd=10 into buffer"
  ↓
kernel:
  выполняет операцию
  ↓
user space:
  CQE: "read completed, result = 128 bytes"
```

3. Почему “ring”

Очереди называются ring, потому что это кольцевые буферы.

```
head → начало чтения
tail → конец записи
```

Когда индекс доходит до конца массива, он возвращается в начало.

```
[0][1][2][3][4][5][6][7]
      ↑       ↑
    head   tail
```

Плюс кольцевого буфера:

не нужно постоянно выделять память
можно быстро добавлять и забирать элементы
удобно разделять память между user space и kernel space

Ключевая идея `io_uring`:

Очереди разделены между приложением и ядром через mmap. Поэтому часть взаимодействия происходит без копирования больших структур через syscall.

4. Основные системные вызовы `io_uring`

Низкоуровнево есть три главных syscall-а:

```
io_uring_setup  
io_uring_enter  
io_uring_register
```

`io_uring_setup`

Создаёт `io_uring` instance.

Примерно:

```
создать SQ  
создать CQ  
вернуть fd объекта io_uring
```

`io_uring_enter`

Сообщает ядру:

```
я положил заявки в SQ  
начни их выполнять  
или подожди завершений в CQ
```

Смысл:

```
submit operations  
wait for completions
```

`io_uring_register`

Позволяет заранее зарегистрировать ресурсы:

файловые дескрипторы
буферы

Зачем?

Чтобы уменьшить накладные расходы при каждой операции.

Например, если буферы заранее зарегистрированы, ядру не нужно каждый раз заново проверять и фиксировать пользовательскую память.

5. Практически используют `liburing`

Вручную работать с syscall-ами неудобно.

Обычно используют библиотеку:

```
#include <liburing.h>
```

Типичный цикл:

```
io_uring ring;  
  
io_uring_queue_init(QueueDepth, &ring, 0);  
  
io_uring_sqe* sqe = io_uring_get_sqe(&ring);  
  
io_uring_prep_read(sqe, fd, buffer, size, 0);  
  
io_uring_submit(&ring);  
  
io_uring_cqe* cqe;  
io_uring_wait_cqe(&ring, &cqe);  
  
int result = cqe->res;  
  
io_uring_cqe_seen(&ring, cqe);
```

Смысл:

get_sqe	→ получить место под заявку
prep_read	→ заполнить заявку
submit	→ отправить заявки ядру
wait_sqe	→ дожждаться завершения
sqe->res	→ результат операции
sqe_seen	→ пометить completion обработанным

6. `epoll` vs `io_uring`: readiness vs completion

`epoll`

ядро: fd готов к чтению
программа: `read(fd, buffer)`

То есть `epoll` говорит:

“Попробуй прочитать, скорее всего получится.”

Но сам `read` делает пользователь.

`io_uring`

программа: прочитай из fd в buffer
ядро: прочитало, вот результат

То есть `io_uring` говорит:

“Операция завершилась, результат такой-то.”

Сравнение:

Механизм	Что сообщает
<code>epoll</code>	fd готов
<code>io_uring</code>	операция завершилась

Это важнейшая концептуальная разница.

7. Пример операции `read` через `io_uring`

В `epoll`:

```
if (event & EPOLLIN) {
    ssize_t n = read(fd, buffer, size);
}
```

В `io_uring`:

```
io_uring_sqe* sqe = io_uring_get_sqe(&ring);

io_uring_prep_read(sqe, fd, buffer, size, 0);

sqe->user_data = reinterpret_cast<uint64_t>(connection);

io_uring_submit(&ring);
```

Потом получаем completion:

```
io_uring_cqe* cqe;
io_uring_wait_cqe(&ring, &cqe);

auto* connection = reinterpret_cast<Connection*>(cqe->user_data);
int result = cqe->res;

io_uring_cqe_seen(&ring, cqe);
```

`cqe->res` — результат операции:

```
> 0  — прочитали столько байт
0    — EOF
< 0  — ошибка, обычно отрицательный errno
```

Важное отличие:

В `io_uring` ошибки обычно приходят как отрицательные значения в `cqe->res`, а не через глобальный `errno`.

Например:

```
cqe->res == -ECONNRESET
cqe->res == -EAGAIN
```

8. `user_data`

Почти всегда в SQE кладут `user_data`.

```
sqe->user_data = ...;
```

Это поле вернётся в CQE.

Зачем?

Чтобы понять, к какому соединению и какой операции относится завершение.

Например:

```
enum class OpType {
    Accept,
    Read,
    Write
};

struct Request {
    OpType type;
    Connection* connection;
};
```

При отправке операции:

```
Request* req = new Request{OpType::Read, connection};

sqe->user_data = reinterpret_cast<uint64_t>(req);
```

При завершении:

```
Request* req = reinterpret_cast<Request*>(cqe->user_data);

if (req->type == OpType::Read) {
    // завершился read
}
```

Это аналог per-connection state в `epoll`, но теперь нужно ещё знать тип завершившейся операции.

9. Сервер через `io_uring`: общая схема

Сервер строится вокруг цикла завершений:

1. Поставить ассерт-заявку.
2. Ждать completion.


```
3. Если завершился accept:
    получить client_fd
    поставить новый accept
    поставить read для client_fd
4. Если завершился read:
    если bytes > 0:
        обработать данные
        поставить write или новый read
    если bytes == 0:
        закрыть соединение
5. Если завершился write:
    если отправили не всё:
        поставить write на остаток
    иначе:
        поставить следующий read
```

Это уже не “fd готов”, а “операция завершилась”.

10. Почему после accept надо поставить новый accept

В `epoll` listening fd постоянно зарегистрирован в `epoll`.

Когда он готов, мы делаем `accept` в цикле.

В `io_uring` мы явно отправляем операцию:

```
accept(server_fd)
```

Когда она завершилась, ядро вернуло один `client_fd`.

Если мы хотим принимать новые подключения, нужно снова поставить accept-заявку.

```
accept completion
  ↓
process client_fd
  ↓
submit next accept
```

В production часто держат сразу несколько pending accept-заявок, чтобы быстрее принимать burst подключений.

11. Упрощённый skeleton `io_uring` echo-сервера

Это не код для запоминания наизусть, а схема.

```

enum class OpType {
    Accept,
    Read,
    Write
};

struct Connection {
    int fd = -1;
    std::string in;
    std::string out;
    std::array<char, 4096> buffer;
};

struct Request {
    OpType type;
    Connection* conn = nullptr;
};

void submit_accept(io_uring& ring, int server_fd) {
    auto* req = new Request{OpType::Accept, nullptr};

    io_uring_sqe* sqe = io_uring_get_sqe(&ring);

    io_uring_prep_accept(sqe, server_fd, nullptr, nullptr, 0);

    sqe->user_data = reinterpret_cast<uint64_t>(req);

    io_uring_submit(&ring);
}

void submit_read(io_uring& ring, Connection* conn) {
    auto* req = new Request{OpType::Read, conn};

    io_uring_sqe* sqe = io_uring_get_sqe(&ring);

    io_uring_prep_recv(
        sqe,
        conn->fd,
        conn->buffer.data(),
        conn->buffer.size(),
        0
    );

    sqe->user_data = reinterpret_cast<uint64_t>(req);

    io_uring_submit(&ring);
}

void submit_write(io_uring& ring, Connection* conn) {
    auto* req = new Request{OpType::Write, conn};

```

```

io_uring_sqe* sqe = io_uring_get_sqe(&ring);

io_uring_prep_send(
    sqe,
    conn->fd,
    conn->out.data(),
    conn->out.size(),
    0
);

sqe->user_data = reinterpret_cast<uint64_t>(req);

io_uring_submit(&ring);
}

```

Цикл обработки:

```

io_uring ring;
io_uring_queue_init(1024, &ring, 0);

submit_accept(ring, server_fd);

while (true) {
    io_uring_cqe* cqe = nullptr;

    io_uring_wait_cqe(&ring, &cqe);

    auto* req = reinterpret_cast<Request*>(cqe->user_data);
    int res = cqe->res;

    if (req->type == OpType::Accept) {
        submit_accept(ring, server_fd);

        if (res >= 0) {
            int client_fd = res;

            auto* conn = new Connection;
            conn->fd = client_fd;

            submit_read(ring, conn);
        }
    }

    if (req->type == OpType::Read) {
        Connection* conn = req->conn;

        if (res > 0) {
            conn->out.assign(conn->buffer.data(), res);
            submit_write(ring, conn);
        }
    }
}

```

```

    } else {
        close(conn->fd);
        delete conn;
    }
}

if (req->type == OpType::Write) {
    Connection* conn = req->conn;

    if (res >= 0) {
        submit_read(ring, conn);
    } else {
        close(conn->fd);
        delete conn;
    }
}

delete req;
io_uring_cqe_seen(&ring, cqe);
}

```

Этот код учебный и упрощённый.

В настоящем сервере надо аккуратно обрабатывать:

```

partial write
ошибки
закрытие соединения
lifetime Connection
несколько pending операций на одно соединение
backpressure
registered buffers

```

12. Partial read/write в `io_uring`

`io_uring` не отменяет реальность TCP.

Даже если операция завершилась, она могла завершиться частично.

Read

```

res > 0:
    прочитали res байт
    это может быть часть сообщения

```

Нужен framing и input buffer.

Write

```
res > 0:
    отправили res байт
    но если out.size() больше, надо отправить остаток
```

То есть `output_buffer` всё равно нужен.

`io_uring` не избавляет от:

```
TCP как поток байтов
частичные операции
framing
backpressure
per-connection state
```

Он меняет способ ожидания и получения результатов операций.

13. Multithreaded server через `io_uring`

Есть несколько архитектур.

Вариант А. Один `io_uring` loop + thread pool для CPU

```
io_uring thread:
    принимает completion events
    делает accept/read/write
    тяжёлую CPU-работу отправляет в thread pool

thread pool:
    обрабатывает CPU-heavy задачи
    возвращает результат в io_uring loop
```

Плюсы:

```
простая модель владения соединениями
меньше гонок за Connection
один поток отвечает за I/O state
```

Минусы:

```
один I/O loop может стать bottleneck
нужен механизм возврата результата в I/O loop
```

Вариант В. Несколько `io_uring` loop-ов

```
worker #1:
    свой io_uring
    свой server_fd или свой набор client_fd

worker #2:
    свой io_uring
    свой server_fd или свой набор client_fd
```

Как распределять соединения?

1. Главный поток делает акцепт и передаёт `client_fd` одному из `io_uring` loop-ов.
2. Или каждый worker имеет свой listening socket через `SO_REUSEPORT`.

Это похоже на многопоточный `epoll`.

С `SO_REUSEPORT`:

```
каждый worker:
    создаёт свой server_fd
    bind на один порт с SO_REUSEPORT
    создаёт свой io_uring
    submit_accept(server_fd)
    обслуживает свои Connection
```

Плюсы:

```
масштабирование по ядрам
каждый Connection принадлежит одному worker-у
меньше синхронизации
```

Минусы:

```
сложнее балансировка
сложнее shutdown
надо аккуратно управлять lifetime объектов
```

14. Почему `io_uring` может быть быстрее `epoll`

1. Меньше системных вызовов

В `epoll` типичный путь:

```
epoll_wait  
read  
write  
epoll_ctl для изменения интересов
```

В `io_uring` можно положить много SQE и одним `io_uring_enter` отправить пачку операций.

Потом получить пачку completions.

```
batch submit  
batch complete
```

Это снижает overhead user ↔ kernel.

2. Completion-based модель

`epoll` говорит:

```
fd готов, сделай read
```

`io_uring` говорит:

```
read завершён, вот результат
```

Это удобнее для асинхронной композиции:

```
accept completed → submit read  
read completed  → submit write  
write completed → submit read
```

3. Единый интерфейс для разных операций

Через `io_uring` можно работать не только с сокетами, но и с файлами, таймерами, connect, accept, send, recv, timeout, open/close и т.д.

То есть event loop может обрабатывать разные типы асинхронных операций в одной модели completions.

4. Возможность регистрации буферов и fd

`io_uring_register` позволяет заранее зарегистрировать ресурсы.

Потенциальные плюсы:

меньше проверок на каждой операции
меньше копирования служебных данных
лучше latency

5. SQPOLL

У `io_uring` есть режим, где kernel thread сам опрашивает submission queue.

Идея:

пользователь кладёт SQE
kernel polling thread замечает заявку
дополнительный syscall может быть не нужен

Плюс:

ниже latency
меньше syscall overhead

Минус:

kernel thread постоянно потребляет CPU
нужно аккуратно использовать

15. Недостатки `io_uring` по сравнению с `epoll`

1. Сложнее API и модель владения

В `epoll`:

`event.data.fd = fd`

В `io_uring`:

нужно создать SQE
назначить user_data
сохранить Request
следить за lifetime буферов


```
следить за lifetime Connection
обработать CQE
```

То есть ошибок по времени жизни объектов становится больше.

2. Сложнее cancellation и shutdown

Если операция уже отправлена в ядро, надо понимать:

```
можно ли её отменить
что делать, если объект закрывается
что делать, если CQE придёт после решения закрыть Connection
```

С `epoll` обычно проще:

```
удалил fd из epoll
закрыл fd
удалил state
```

В `io_uring` pending операции могут завершиться позже.

3. Не все операции одинаково “асинхронны”

Некоторые операции могут внутри ядра всё равно быть выполнены через worker threads или иметь ограничения.

Поэтому нельзя думать:

```
io_uring магически делает всё бесплатным
```

Он снижает overhead, но не отменяет физическую стоимость I/O.

4. Linux-specific и зависит от версии ядра

`epoll` тоже Linux-specific, но старше и распространённое.

`io_uring` моложе, активно развивался, и поведение/набор поддерживаемых операций сильнее зависит от ядра.

5. Security / complexity

Из-за сложности и богатого API `io_uring` имеет большую поверхность для ошибок и ограничений.

В некоторых окружениях его могут отключать или ограничивать.

6. Не всегда быстрее

Для простого TCP echo-сервера на небольшой нагрузке `epoll` может быть достаточно быстрым и проще.

`io_uring` выигрывает, когда важны:

много операций
batching
снижение syscall overhead
унификация network + file I/O
низкая latency

Но сложность выше.

16. `epoll` vs `io_uring`

Свойство	<code>epoll</code>	<code>io_uring</code>
Модель	readiness	completion
Паттерн	reactor	proactor
Что возвращает	fd готов	операция завершилась
После события	сам вызываешь read/write	уже есть результат операции
API	проще	сложнее
Syscall overhead	выше при большом числе операций	ниже за счёт ring/batching
Работа с файлами	не универсальный async I/O	шире и универсальнее
Lifetime объектов	проще	сложнее из-за pending операций
Portability	Linux-specific	Linux-specific, но новее
Хорош для	сетевой readiness-loop	асинхронные операции и batching

17. Важная фраза: `io_uring` не отменяет event loop

Иногда кажется:

“Если `io_uring` асинхронный, event loop больше не нужен.”

На самом деле нужен.

Просто event loop теперь ждёт не readiness events, а completion events.

```
epoll event loop:
    wait ready fd
    do read/write

io_uring event loop:
    wait completed operation
    decide next operation
    submit next SQE
```

То есть логика соединения всё равно превращается в state machine.

18. Что сказать на экзамене: короткая версия

Можно отвечать так:

`io_uring` — современный Linux-интерфейс асинхронного I/O. Он использует две кольцевые очереди, разделяемые между user space и kernel space: submission queue и completion queue. Пользователь кладёт в submission queue заявки — SQE, например `accept`, `recv`, `send`. Ядро выполняет операции и кладёт результаты в completion queue в виде CQE.

Главное отличие от `epoll`: `epoll` — readiness-based reactor. Он говорит: “fd готов, теперь сам вызови `read` или `write`”. `io_uring` ближе к proactor-модели: пользователь отправляет конкретную операцию, например “прочитай из fd в buffer”, а потом получает completion: “операция завершилась, прочитано N байт”.

Сервер через `io_uring` строится как цикл завершений. Сначала мы ставим `accept`-заявку. Когда приходит CQE по `accept`, получаем `client_fd`, сразу ставим следующий `accept` и ставим `read/recv` для клиента. Когда завершается `read`, если прочитали данные, формируем ответ и ставим `write/send`. Когда завершается `write`, ставим следующий `read`. Для связи completion с соединением используют `user_data`, куда кладут указатель на Request или Connection.

Многопоточно можно сделать несколько `io_uring` loop-ов, у каждого свой ring и свой набор соединений. Соединения можно распределять главным `accept`-потокom или через `SO_REUSEPORT`, когда у каждого worker-а свой listening socket. Важно,

чтобы одно соединение обычно принадлежало одному loop-у, иначе нужно много синхронизации.

Преимущества по сравнению с `epoll`: меньше системных вызовов за счёт кольцевых буферов и batching, completion-based модель, единый интерфейс для разных I/O-операций, возможность регистрировать буферы и fd. Недостатки: API сложнее, сложнее lifetime объектов и cancellation, pending операции могут завершиться после решения закрыть соединение, зависимость от Linux и версии ядра, не всегда выигрыш на простой нагрузке.

19. Мини-код для ответа на доске

```
io_uring ring;
io_uring_queue_init(1024, &ring, 0);
```

Поставить accept:

```
void submit_accept(io_uring& ring, int server_fd) {
    auto* req = new Request{OpType::Accept, nullptr};

    io_uring_sqe* sqe = io_uring_get_sqe(&ring);

    io_uring_prep_accept(sqe, server_fd, nullptr, nullptr, 0);

    sqe->user_data = reinterpret_cast<uint64_t>(req);

    io_uring_submit(&ring);
}
```

Цикл completions:

```
submit_accept(ring, server_fd);

while (true) {
    io_uring_cqe* cqe = nullptr;

    io_uring_wait_cqe(&ring, &cqe);

    auto* req = reinterpret_cast<Request*>(cqe->user_data);
    int res = cqe->res;

    if (req->type == OpType::Accept) {
        submit_accept(ring, server_fd);

        if (res >= 0) {
            auto* conn = new Connection;
```

```

        conn->fd = res;
        submit_read(ring, conn);
    }
}

if (req->type == OpType::Read) {
    Connection* conn = req->conn;

    if (res > 0) {
        conn->out.assign(conn->buffer.data(), res);
        submit_write(ring, conn);
    } else {
        close(conn->fd);
        delete conn;
    }
}

if (req->type == OpType::Write) {
    Connection* conn = req->conn;

    if (res >= 0) {
        submit_read(ring, conn);
    } else {
        close(conn->fd);
        delete conn;
    }
}

delete req;
io_uring_cqe_seen(&ring, cqe);
}

```

Что объяснить:

SQE – заявка на операцию
 CQE – завершение операции
 user_data связывает completion с Connection/Request
 accept completion → submit next accept + read
 read completion → process + write
 write completion → next read
 cqe->res содержит результат операции

20. Мини-проверка

1. Чем `io_uring` концептуально отличается от `epoll`?
2. Что такое Submission Queue?
3. Что такое Completion Queue?
4. Что такое SQE?

5. Что такое CQE?
6. Зачем нужно поле `user_data`?
7. Что делает `io_uring_submit`?
8. Что обычно означает `cqe->res`?
9. Как выглядит схема echo-сервера через `io_uring`?
10. Как можно сделать многопоточный сервер через `io_uring`?
11. Назови преимущества `io_uring` перед `epoll`.
12. Назови недостатки `io_uring` по сравнению с `epoll`.